06. MVP slice and migration

Status	Draft v1.0 for team review. Aligned to `architecture.md`: Core owns commerce; the vendor-admin service is the seller back-office
Date	2026-10-20
Depends on	`02` to `05`, `architecture.md`
Deadline	Code freeze Oct 30, buffer to Nov 2

Legend: `[As-is]` verified in today's code, `[MVP]` built by Oct 30, `[Target]` after the MVP, `[Assumption]` needs a team decision.

1. Principle

Ship the **core loop** by Oct 30 on the hardened Core service plus the new vendor-admin service:

```
vendor applies, admin approves, vendor lists products and sets stock, admin approves them,
customer swipes, customer places an order (cash or UPI on delivery), vendor confirms and ships,
courier delivers, ops marks delivered and collected
```

Do not build the target infrastructure (GraphQL, Redis feed queues, a worker, object storage, **online payments switched on**) for the MVP. Make choices now that keep the later migration mechanical: the same field names and enums as `04`, the same status values as `02`, the same routes as `05`.

2. MVP scope

Capability	MVP build	Where	Target equivalent	Register IDs closed
Safe access	Token login for the app; token on every route except a public list; guest-only upgrade; admin-only sensitive routes; `DEBUG` off; no password logging	Core: `api/views/user_views.py`, `settings.py`, `urls.py`, app `shared` client	`auth` module	SEC-01 to SEC-06, SEC-08, SEC-09, SEC-11, SEC-12
Vendor application	Website form posts to the vendor-admin API; vendor row `applied`	Vendor-admin `vendors`	Same	CAT-04 (seller login fix)
Vendor portal and admin	React portal (pages in section 5); SQLAdmin data screens; admin order pages	Portal repo `frontend/`, `backend/`	Ops screens	n/a
Product listing and publishing	Product form with variants, 3 or more photos, fabric, return policy; submit; admin review; publish event to Core; embedding in Core	Vendor-admin `catalog`; Core `catalog`	Same with pgvector	CAT-01 (`price_value`), AI-08
Stock	Vendor sets `on_hand` in the portal; the vendor-admin service sends it to Core; Core owns the count and reserves at checkout	Core `orders` (inventory), vendor-admin command client	Same	n/a
Feed	Fixed learning, cosine ranking, 7 plus 3 slate, filters for price, fabric, product type; only items with available stock	Core `recommendation_views.py`	`feed` module with pgvector	AI-01, AI-02, AI-03, AI-09, AI-12 to AI-17 (main path)
Checkout	One Core transaction: server prices, atomic stock reservation, one order per vendor, idempotency key, events	Core `orders`	Same	COM-01 to COM-06, COM-12
Fulfilment	Vendor confirm, cancel, ship; admin marks delivered and payment collected; each is a command from vendor-admin to Core	Vendor-admin `orders` client; Core internal API	Courier webhook	n/a
Timers	Core auto-cancels unconfirmed orders after 4 business hours (reminder first); releases expired reservations	Core cron commands	Worker	n/a
Notifications	Email to vendors (vendor-admin); push to customers (Core); WhatsApp by click-to-chat links used by ops	Both	WhatsApp provider	n/a
Payments switch	Razorpay create order, signed webhook, refund, built in **test mode** behind `online_payments_enabled`; **off at launch**	Core `orders`, `integrations`	Switch on after approval	n/a
Images	Vendor images upload straight to Cloudinary; the app loads Cloudinary URLs directly	Vendor-admin, app image helper	Object storage and CDN	CAT-05, CAT-11, CAT-12
Service contract	`contracts/` folder in the portal repo (Core pins a copy): internal OpenAPI and event schemas; signed calls; outbox and inbox	Both repos	Broker if needed	n/a
Deep links	Website landing pages for `/closet/:id` and `/product/:id`, iOS link file	Website repo	Same	C8 in `02`
Release	Two Docker images (one per repo) on one small VM with Docker Compose and Caddy; staging; nightly backups; Play internal testing then staged rollout	`Dockerfile`s, `infra/`	Managed Postgres, second VM	CAT-06

Explicitly not in the MVP: online payments switched **on**, GraphQL, pgvector, Redis feed queues, a worker, multi-interest taste, the agentic salesperson, vendor staff beyond owner and staff roles, automatic payouts, the Expo migration (a gated experiment only), TypeScript rewrite of the app.

3. Data changes `[MVP]`

The "target" column is the table in `04-database-design.md`.

Core database (Django migrations)

Current model	Change	Target
`Item` and `ItemVariant`	Become the catalog copy: add `source` (`vendor`, `scraped`; existing rows `scraped`), `status`, `vendor_id`, `vendor_name`, `commission_bps`, `version`, `feed_eligible`, `fabric`, `returnable`, `return_window_days`; `price_value` set from the cheapest active variant and backfilled; variant `price` becomes `price_paise` (integer); fix `__str__`	`catalog_items`
(new) `Inventory`, `StockReservation`	Stock counters and reservations	`inventory`, `stock_reservations`
`Order`	Reshape: `vendor_id`, `status`, `payment_method`, `payment_status`, `number` (unique), money fields, `commission_bps`, `commission_paise`, `price_versions`, `customer_name`, `customer_phone`, `checkout_group_id`, timers, `idempotency_key`, `version`	`orders`
`OrderItem` (new, replaces the many-to-many)	`order`, `variant_id`, `product_id`, `title`, `size`, `color`, `image_url`, `unit_price_paise`, `quantity`	`order_items`
`OrderStatusEvent`, `Payment` (new; provider `none` at launch)	History and payment rows	`order_status_events`, `payments`
`OutboxEvent`, `InboxEvent`, `IdempotencyKey`, `RefreshToken` (new)	Events, de-duplication, tokens	Same names in `04`
`Action.like_status`	Normalise to `like`, `dislike`, `save`, `cart`, `order` (data migration: `"True"`, `"False"`, `"1.5"`, `"2"`); update `get_liked_items`	`swipe_events`
`User`	Make `email` and `phone_number` unique where set (after cleaning rows where the phone was stored in `email`); keep `is_shadow` (renamed `is_guest` in the target)	`users`
`Brand`	Copied once to the vendor-admin database, then retired from Core	`vendors` (vendor-admin)

Existing `Cart`, `Closet`, `Address` models stay as they are in the MVP.

Vendor-admin database (`shazlo_vendor`, new, Alembic): `vendors` (with `status`, `city`, `whatsapp_number`, `commission_bps` default 1500, agreement fields, return policy), `commission_rates`, `vendor_users`, `admin_users`, `refresh_tokens`, `vendor_policies`, `categories`, `products`, `product_variants`, `product_images`, `product_status_events`, `vendor_orders`, `vendor_order_items`, `outbox_events`, `inbox_events`, `idempotency_keys`, `vendor_notification_log`, `audit_log` (columns in `04`). Added after the core loop works: `vendor_locations`, `vendor_status_events`, `import_jobs`, `vendor_payouts`, `payout_lines`.

Migration safety: each change is a separate migration; data migrations are repeatable; back up the database before the migration run; verify counts after.

4. MVP API `[MVP]`

The full design is `05-api-design.md`; this section is the MVP subset on **Core** (REST under `/v1`).

Changed routes (token required, `user_id` no longer trusted)

Route	Change
`POST /v1/auth/shadow`	Returns `{access, refresh, user}`; rate limited
`POST /v1/auth/signup`	Requires the guest's bearer token; refuses non-guests; clears the guest flag
`POST /v1/auth/login`, `POST /v1/auth/google-login`	Return tokens; no password logging
`POST /v1/auth/refresh`, `POST /v1/auth/logout`	New
`POST /v1/auth/delete`	Token required; deletes or anonymises; emits `customer.deleted`
`POST /v1/items/getinitial`	Body no longer carries `userid`, `preference_vector` or `exclude_ids`; returns the 7 plus 3 slate without embeddings
`POST /v1/user/swipes`	Body: `item_id`, `action`, `dwell_ms`, `client_event_id`; no vectors; learning happens on the server
Closets, cart, address routes	Token identifies the user; closet writes check membership

New routes on Core

Route	Purpose	Notes
`GET /v1/public/products/{id}`, `GET /v1/public/closets/{id}`, `GET /v1/public/config`, `GET /v1/serviceability?pincode=`	Website previews, flags (including `online_payments_enabled`), launch pincodes	Closet-id links kept in the MVP so existing shared links work
`POST /v1/orders/`	Place an order (one transaction)	Header `Idempotency-Key` required; returns one order per vendor; reserves stock
`GET /v1/orders/`, `GET /v1/orders/{id}/`	The customer's orders	Core's own tables
`POST /v1/orders/{id}/cancel/`	Cancel from `placed` or `confirmed`	Releases stock
`POST /v1/webhooks/razorpay`	Payment webhook	Inactive while the switch is off

Route	Purpose	Notes
`POST /internal/v1/...`	Commands and events from the vendor-admin service	Not public; signed; `05` section 8

Locked or removed: `order/all`, `items/create`, `items/find_dups` (move to management commands or admin), `order`, `razorpay-checkout/`, `verify-payment/`, `order-db/` (replaced by the new order route and the webhook), and the image proxy (`items/getimage`) is hardened (allow-list, timeout, cache headers) or bypassed for Cloudinary images.

Vendor-admin routes (vendor, admin, vendor application, internal receiver) are new and listed in `05` sections 4 to 6 and 8.

5. Vendor portal pages `[MVP]` (React, vendor-admin API)

Page	Purpose
`/login`, `/logout`, `/password/forgot`, `/password/reset`	Sign in with email or phone; rate limited
`/`	Dashboard: status badge, orders waiting for confirmation, products in review
`/products`	List with status filter and stock
`/products/new`, `/products/{id}`	Product form: title, description, product type, gender, fabric, return policy, variants (size, price, MRP), stock per size
`/products/{id}/images`	Upload three or more photos with an angle each (direct to Cloudinary)
`/products/{id}/submit`	Send for review
`/products/import`	CSV import with per-row errors (stretch goal, only if the core pages are done by Oct 27)
`/orders`, `/orders/{id}`	Orders (from the read copy) with Confirm, Cancel (reason) and Ship buttons, each a command to Core
`/policy`, `/profile`	Return policy, contact details
`/statements`	Monthly list of delivered orders, commission and net amount
`/admin/*` (admin accounts only)	Vendor review queue, product review queue, order operations (mark delivered, mark payment collected, cancel, refund), failed events and resync

Security: vendor-admin tokens, login throttling, CORS limited to `seller.shazlo.store`, object checks on every call (a vendor sees only its own rows), commands carry the actor id into Core's audit log.

6. Feed logic `[MVP]` (Core)

1. **Eligible products:** `feed_eligible` (approved, a variant with available stock, an embedding present), `price_value` set, and not already acted on by this user (read from the actions table, not the JSON list). Link-out scraped items skip the stock test.
2. **Learning (server side):** each swipe updates the user's single taste vector: `u = normalize(u + eta * weight * e)`, where `e` is the product embedding minus the catalog mean, `eta = max(0.05, 1 / (n + 2))`, `n` is the user's swipe count. Weights: `like` +1, `dislike` -0.4, `save` +2, `cart` +3, `order` +5 `[Assumption: tune]`.
3. **Score:** cosine similarity between the (centered) taste vector and each eligible product, after the user's filters (gender, price, fabric, product type, brand).
4. **Slate of 10:** group the top 200 by category; pick the category with the highest average of its top 5 scores; take 7 from it; take 3 more from three different other categories (best score each). Interleave the exploration items at positions 4, 7 and 10 `[Assumption]`. Attach `explanation` text.
5. **Cold start:** with no usable vector, use the most-liked products per category plus a few random ones.
6. **Not used any more for serving:** the SGD model, `saved_model.pkl`, `train_count.txt`.
7. **Memory:** keep embeddings as compact arrays and limit the in-memory cache to eligible products so the web process stays within its memory limit.

7. Background jobs and notifications `[MVP]`

No worker. Scheduled commands in each service, run by the VM's cron:

Service	Command	Frequency	Purpose
Core	`embed_approved_products`	every 5 minutes	Create embeddings for newly published products (retry on failure)
Core	`remind_unconfirmed_orders`	every 15 minutes	Emit `order.confirmation_overdue` so the vendor-admin service emails the vendor
Core	`autocancel_unconfirmed_orders`	every 30 minutes	Cancel orders unconfirmed after 4 business hours, release stock, push to the customer
Core	`expire_reservations`	every 5 minutes	Release stock held by orders past `reservation_expires_at` and cancel unpaid online orders (when the switch is on)
Core	`outbox_publish`	every minute	Deliver events to the vendor-admin service
Core	`reconcile_catalog`	nightly	Compare versions with the vendor-admin service and request re-sends
Core	`cleanup_guests`	daily (optional)	Remove old guest accounts with no activity
Vendor-admin	`outbox_publish`	every minute	Deliver events to Core
Vendor-admin	`reconcile_orders`	nightly	Compare versions with Core and request re-sends

Notifications:

- **Vendor:** email on every new order and on the overdue reminder (vendor-admin); a banner in the portal; ops can send a WhatsApp message from the admin pages using a prefilled click-to-chat link (`wa.me/<number>?text=<summary>`).
- **Customer:** push for status changes (Core); ops sends WhatsApp updates by click-to-chat.
- Automatic WhatsApp messages need an approved WhatsApp Business provider and templates, which take time; treat them as `[Target]` unless the approval arrives early `[Assumption]`.

Stock: **reserved at order placement** (atomic, in Core), committed on ship, released on cancel, expiry or unpaid timeout. Two customers cannot order the last item.

8. Hosting and release `[MVP]`

- A Dockerfile per service (one in each repo) and a production `docker compose` in `shaz/infra` for the single small VM: Caddy, core, vendor, Postgres with two databases; Cloudflare in front; each repo's CI pushes its image and redeploys its own service. Staging is a second Compose project with its own databases. Nightly `pg_dump` of both databases to object storage with a weekly restore test (`architecture.md` section 9).
- Cloudflare: admin restricted by Access; docs and schema hidden in production; a cache rule for any image proxy that remains.
- Secrets in the host's environment settings; rotate anything that was committed; `.env.example` added. Razorpay and its webhook secret exist only in Core's settings.
- Android release: bump `versionCode` above 19 (and `versionName`); upload to Play **internal testing**, test, then a **staged rollout**; the upload keystore must be handed over (see the handover steps in the earlier plan).
- Backend and app are released together because request bodies change; the old app version loses access once the token requirement is on, so the Play rollout is staged and a minimum-version check comes from `/v1/public/config`.

9. App and website changes `[MVP]`

App: use tokens and a single API client; stop sending vectors; show the Cart (re-enable the commented tab) and order screens (address, payment method from `publicConfig`, confirmation, order list); show vendor products as buyable and scraped products as link-out; filters for price, fabric and product type; load Cloudinary images directly; handle `stock_unavailable` per line; fix the crashes on the main path (end of deck, missing sizes, stuck delete overlay, duplicate feed load, double next-card); API URL from build configuration; reconnect the payment screen to the new Core flow only when the switch is turned on.

Website (details in `07-website-audit-and-plan.md`): vendor funnel and application form (posts to the vendor-admin service), landing pages for `/closet/:id` and `/product/:id`, iOS link file, working contact details, fixed legal pages, SEO tags, secrets removed from the repository.

10. Migration to the target `[Target]` (after launch)

Order, each step shippable on its own:

1. **Foundations:** CI hardening, WAL archiving, alerts tuned, runbooks tested.
2. **Payments on:** after Razorpay approves the account, turn the switch on in stages (staff first); decide the vendor money flow (Q37).
3. **Feed:** pgvector queries, precomputed queues (needs Redis and a worker).
4. **Accounts and auth:** move refresh tokens to Redis; admin second factor.
5. **Scale:** managed Postgres, second VM for the vendor-admin service, image CDN.
6. **App cutover:** the app switches to GraphQL behind a remote flag, with a fallback to REST; old REST routes stay until the minimum supported app version is raised.
7. **Retire:** the old Django routes, the SGD model files, the legacy Node and Mongo service, the Cloudinary dependency if object storage replaces it.

Preconditions: tests for each module, row-count reconciliation scripts, a rollback plan per step.

11. Schedule (three tracks) `[Assumption: owners to confirm]`

Dates	Track 1: Core hardening and commerce	Track 2: Vendor-admin service and portal	Track 3: App and feed
Oct 20 to 22	Security fixes, tokens, Docker; **agree the internal contract with Track 2**; inventory and order tables	FastAPI skeleton, database, CI, vendor and product tables; OpenAPI draft	Learning-weights fix; top crashes; (optional) Expo experiment with a gate
Oct 23 to 27	Checkout with stock reservation, order state machine, events; catalog copy receiver; embedding job	Vendor application, login, product form, moderation; `product.published` events; portal login and product pages	Slate logic; app on tokens; cart reachable
Oct 28 to 30	Internal commands (confirm, ship, cancel, deliver, collected), timers, push; staging then production deploy; monitoring	Orders page (copy and commands), stock page, SQLAdmin, vendor email; end-to-end test with real vendors	Order screens; filters; Android internal testing then staged rollout
Oct 31 to Nov 2	Buffer; payments switch in test mode	Buffer; statements	Buffer; iOS build if Android is stable

Feature freeze: Tuesday Oct 27. Cut order if behind: payments switch, CSV import, statements and payouts, iOS build, nightly reconciliation (keep the manual resync), extra filters, hosting extras, the Expo experiment, TypeScript work.

12. Definition of done (Oct 30)

- [ ] No route works without a token (except the public list); the security fixes are in; no secrets in git; passwords not logged.
- [ ] A vendor can apply from the website, an admin approves, and the vendor logs in to the portal.
- [ ] A vendor can add a product with variants and 3 or more photos, set stock, and submit it; an admin approves it; it appears in the feed.
- [ ] Swiping changes later feeds (a test with two different test users gives different slates).
- [ ] A customer can place an order; stock is reserved; the vendor sees it and is emailed; the vendor confirms and ships through the portal (commands to Core); ops marks delivered and collected; the customer gets pushes.
- [ ] Prices and commission come only from Core's catalog copy; two parallel buyers of the last unit cannot both succeed.
- [ ] Checkout works with the vendor-admin service stopped.
- [ ] Payments switch is off, tested in Razorpay test mode on staging, and Razorpay credentials exist only in Core.
- [ ] The server stays up (no sleeping) and handles about 20 concurrent users in a test.
- [ ] `docker compose up` works for a new developer; a restore of both databases into staging has been tested.
- [ ] Shared links open the right page with and without the app installed.
- [ ] A real-phone test from install to delivered order has no crashes.

13. Risks

Risk	Mitigation
Core carries commerce, so Track 1 is the largest	Rebalanced tracks; checkout is one transaction; cut order above
Two new codebases (service and portal) by Oct 30	Contracts first, generated clients, narrow MVP, cut order
Token requirement locks out old app versions	Staged rollout; minimum-version check; release backend and app together
WhatsApp automation is not approved in time	Email plus ops click-to-chat as the MVP path
Admin order pages take longer than planned	Ops use a short script or the internal API for the first vendors; SQLAdmin for data screens
No upload keystore for Play	Handover this week; Play App Signing key reset as the fallback
Embedding service slow or down	Scheduled retry; products remain approved but hidden until embedded
Single VM	Backups, tested restore, alerts, rebuild runbook